Apuntado

Diseño detallado, arquitectura de información y gobernanza

Segundo entregable del proyecto

Juan José Alzate Rojas
Estudiante de prácticas
Fundamentos de Arquitectura de Software
Universidad Nacional de Colombia

El sistema ofrece Apuntado con rondas acumulativas y una modalidad Clásica de partida única. Este informe concreta su arquitectura, el modelo relacional, las decisiones de diseño, los riesgos y las reglas ejecutables de gobernanza.

El backend se organiza como monolito modular con estrategias versionadas y persistencia PostgreSQL; el frontend se despliega de forma independiente. La base de código adjunta implementa y prueba el cierre de ronda y sus límites arquitectónicos; la integración de la aplicación multijugador completa continúa en el MVP.

Repositorio del entregable

https://github.com/AlzateJJ/apuntado-arquitectura

El repositorio independiente apuntado-arquitectura contiene los modelos editables, ADR-01 a ADR-05, esquema SQL, pruebas de arquitectura y pipeline. Los comandos de reproducción se incluyen al final del informe.

Contenido	Página
1. Selección y justificación del estilo arquitectónico	2
2. Arquitectura de información y topología de datos	6
3. Registro formal de decisiones arquitectónicas	13
4. Análisis de riesgo arquitectónico y Risk Storming	16
5. Gobernanza y funciones de idoneidad	20

1. Selección y Justificación del Estilo Arquitectónico

1.1 Evaluación Comparativa de Estilos

El Top 3 del primer entregable es Configurability, Testability y Responsiveness. La extensibilidad complementa la configuración: cambiar un parámetro admitido no equivale a crear una familia nueva de reglas. La integridad y la privacidad restringen cualquier alternativa.

Criterio	Microkernel / plug-ins	Monolito modular
Configurabilidad	Un registro selecciona extensiones. Requiere contratos y validación de compatibilidad.	Perfiles versionados seleccionan estrategias. Las reglas de cada partida permanecen fijas.
Facilidad de pruebas	Permite probar cada plug-in; también exige pruebas de carga, compatibilidad y aislamiento.	Evalúa reglas puras sin red ni base de datos. La coordinación usa puertos sustituibles.
Capacidad de respuesta	Un plug-in local puede ser rápido; aislarlo en otro proceso introduce comunicación.	Las llamadas internas son locales; el cierre usa una transacción. Debe controlarse la CPU compartida.
Extensión y despliegue	Adecuado para extensiones con ciclo de vida independiente. Carga dinámica y seguridad aumentan el esfuerzo.	Una nueva estrategia se registra y se despliega con el conjunto. El núcleo permanece estable.
Costo individual	Más mecanismos para versionar, cargar y retirar extensiones de los que exige el caso.	Menos infraestructura y una frontera transaccional local, a cambio de disciplina de módulos.

Se selecciona el monolito modular. Las dos modalidades pertenecen al mismo dominio, comparten turnos y cartas, y no necesitan extensiones instalables por terceros. Strategy y un registro de implementaciones satisfacen la variabilidad actual. La elección reduce coordinación operativa, pero no garantiza tiempos de respuesta sin medirlos.

Se reconsiderará Microkernel si aparecen extensiones de terceros o despliegues independientes. WebSockets es un mecanismo de transporte; su presencia no convierte el sistema en una arquitectura dirigida por eventos.

1.2 Descripción del Estilo Seleccionado y Topología

Un servidor autoritativo contiene los módulos de sala, coordinación, reglas, consulta y sincronización. Persistencia y transporte implementan contratos del sistema. El navegador envía intenciones; el servidor decide validez, turno y puntuación. La topología inicial contempla frontend independiente en Vercel, backend modular en Railway y una base PostgreSQL central. Monolito modular describe la organización y el despliegue conjunto del backend.

Aspecto	Clásica / partida única	Apuntado
Duración	Una ronda. El primer cierre válido finaliza la partida y conduce a la vista principal.	Varias rondas con puntuación acumulada.
Perfil inicial	classic@1: cierre 4-3-3, conforme a la modalidad Clásica de la guía.	apuntado@1: 4-3-3, 5-5, diez pintas y toque.
Puntuación	Sin movimientos de puntaje ni eliminación acumulativa.	+10 por combinaciones; +25 por pintas; toque sin premio. Los sobrantes se descuentan.
Fin	Ganador del cierre; no se crea otra ronda.	Victoria en +51 o más, o único activo. Eliminación en -101 o menos.
Reglas compartidas	Dos mazos, 104 cartas, sin jokers, diez cartas al evaluar, hasta seis participantes.	Identidad física, grupos y escaleras; as bajo o alto y valor de 10 puntos.

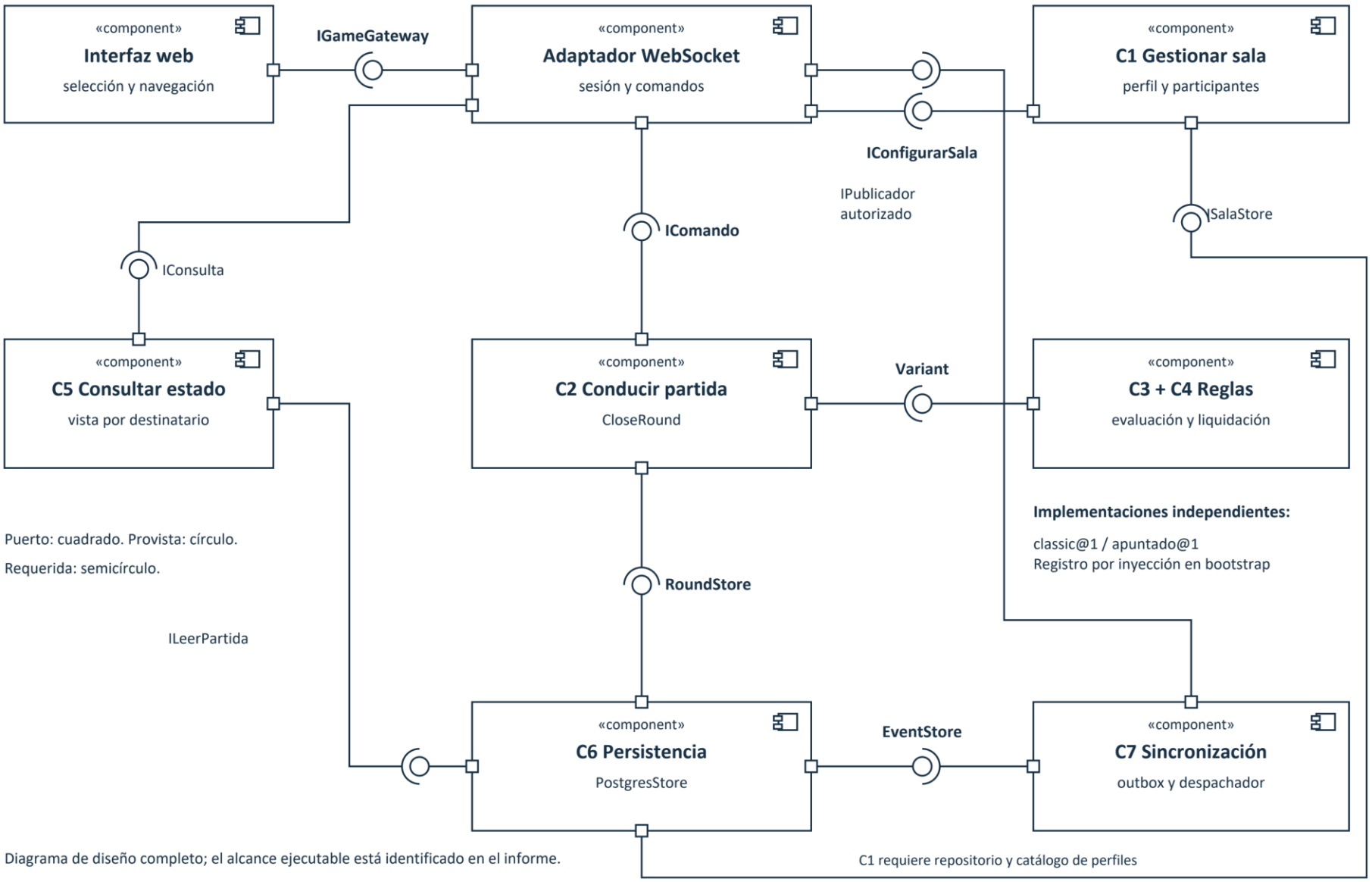
El perfil Clásica v1 adopta 4-3-3 como alcance inicial de diseño; los otros cierres se habilitan en Apuntado. La selección y la versión se fijan al crear la partida. Una ampliación de los cierres de Clásica se publicaría como otro perfil, sin cambiar partidas iniciadas.

Empaquetado y extensión

design/src contiene contracts, domain, application, variants, persistence, synchronization, ui y bootstrap. Las variantes solo conocen contratos y utilidades puras. El punto de composición bootstrap/catalog.ts importa sus implementaciones; CloseRound recibe el catálogo por constructor.

Una variante nueva implementa Variant, incorpora sus pruebas y se registra con otra clave o versión. Cambian el registro y el paquete de reglas; no cambia el coordinador. El despliegue sigue siendo conjunto. Esta distinción evita confundir un patrón Strategy con un microkernel dinámico.

1.3 Diagrama UML de Componentes Detallado



1.3 Contratos y correspondencia con el código

Componente	Contrato y responsabilidad	Alcance ejecutable
C1 Gestionar sala	IConfigurarSala: selecciona perfil, valida participantes y prepara rondas. Requiere repositorio y catálogo.	Modelado; flujo completo de sala pendiente de integración.
C2 Conducir partida	IComando; coordina identidad, turno, fase, versión y cierre atómico mediante RoundStore.	application/closeRound.ts.
C3 Evaluar jugadas	Variant utiliza utilidades de grupos, escaleras, particiones y sobrantes.	domain/ y variants/.
C4 Liquidar ronda	La estrategia calcula cambios y fin de partida sin persistir.	settle de cada variante; resultado tipado Settlement.
C5 Consultar estado	IConsulta: proyecta solo la mano autorizada antes del cierre. El lector SQL se integra por puerto.	application/view.ts sobre un estado recibido.
C6 Persistencia	RoundStore; confirma estado, acta, movimientos, operación y outbox.	persistence/postgres.ts y adaptador de prueba en memoria.
C7 Sincronización	EventStore y AuthorizedPublisher; entrega posterior al commit.	dispatchPending y prueba con publicador sustituible.
Interfaz y transporte	IGameGateway; identidad proveniente de sesión WebSocket y navegación por resultado.	ui/navigation.ts. Servidor WebSocket y pantalla completa pendientes.

El diagrama representa el diseño completo. El código verifica una sección funcional del cierre, no un reemplazo terminado del prototipo. C3 y C4 comparten el paquete de reglas por variante, pero conservan funciones de evaluación y liquidación separadas del coordinador.

IComando corresponde a CloseRound.execute; Variant es el contrato de extensibilidad; RoundStore define la unidad de trabajo. Los nombres ISalaStore e ILeerPartida expresan puertos previstos para completar sala y consultas. La dirección de una llamada no obliga a que el núcleo importe un adaptador concreto.

2. Arquitectura de Información y Topología de Datos

2.1 Justificación de la Topología

Se adopta una topología relacional centralizada en PostgreSQL 18. El servidor concentra la escritura y coordina cada transición de una partida. Los clientes nunca se conectan directamente a la base. Cada partida puede bloquearse sin bloquear deliberadamente las demás.

Alternativa	Ventaja para el caso	Costo o límite
SQL centralizado	PK/FK, consultas de historial y transacción local para acta, puntos y evento.	Administración del servicio, respaldos y posible contención por partida.
SQLite embebido	Instalación sencilla para un proceso y trabajo local.	Concurrencia de escritores más limitada y dificultad al compartir estado entre varias instancias.
Documentos agregados	Una partida incrustada puede actualizarse de forma atómica.	Historial creciente, información duplicada y restricciones de pertenencia menos explícitas.

La consistencia del marcador pesa más que una instalación mínima: un cierre parcial cambia el resultado del juego. Las transacciones SQL y las claves compuestas permiten expresar pertenencia a partida y unicidad de liquidación. La centralización conserva simplicidad frente a una coordinación distribuida, aunque no aporta alta disponibilidad por sí sola.

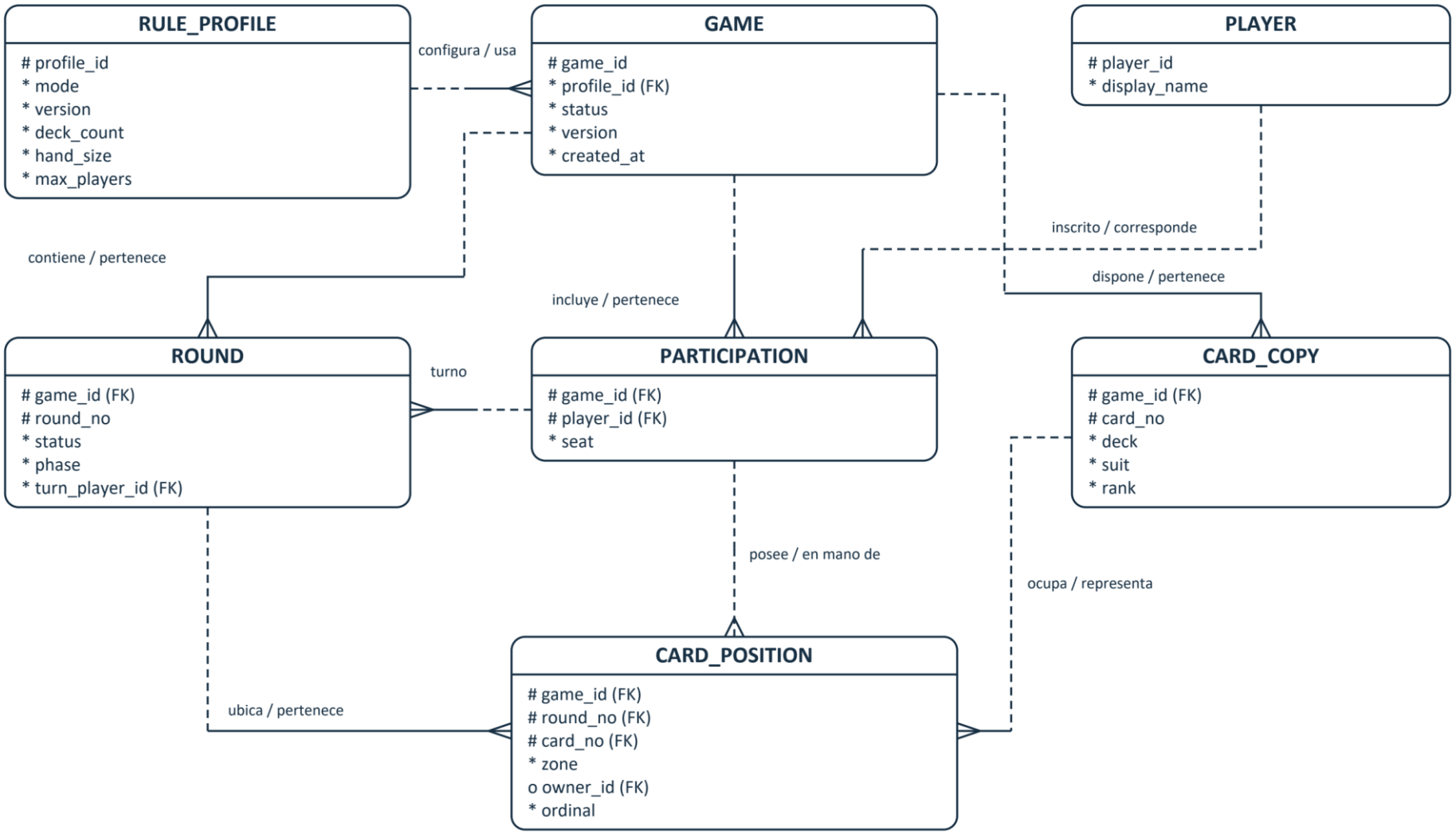
2.2 Componente de Información — Opción A Relacional SQL

El modelo autoritativo utiliza once tablas y una vista derivada. Las dos vistas Barker siguientes forman un único modelo. Se repiten los identificadores de algunas entidades para mantener legibilidad; no representan tablas adicionales.

La identidad de una carta es (game_id, card_no). La combinación (game_id, deck, suit, rank) también es única. Dos ases del mismo palo en mazos distintos son copias válidas; repetir una misma copia en dos posiciones de la misma ronda se impide con la PK de card_position.

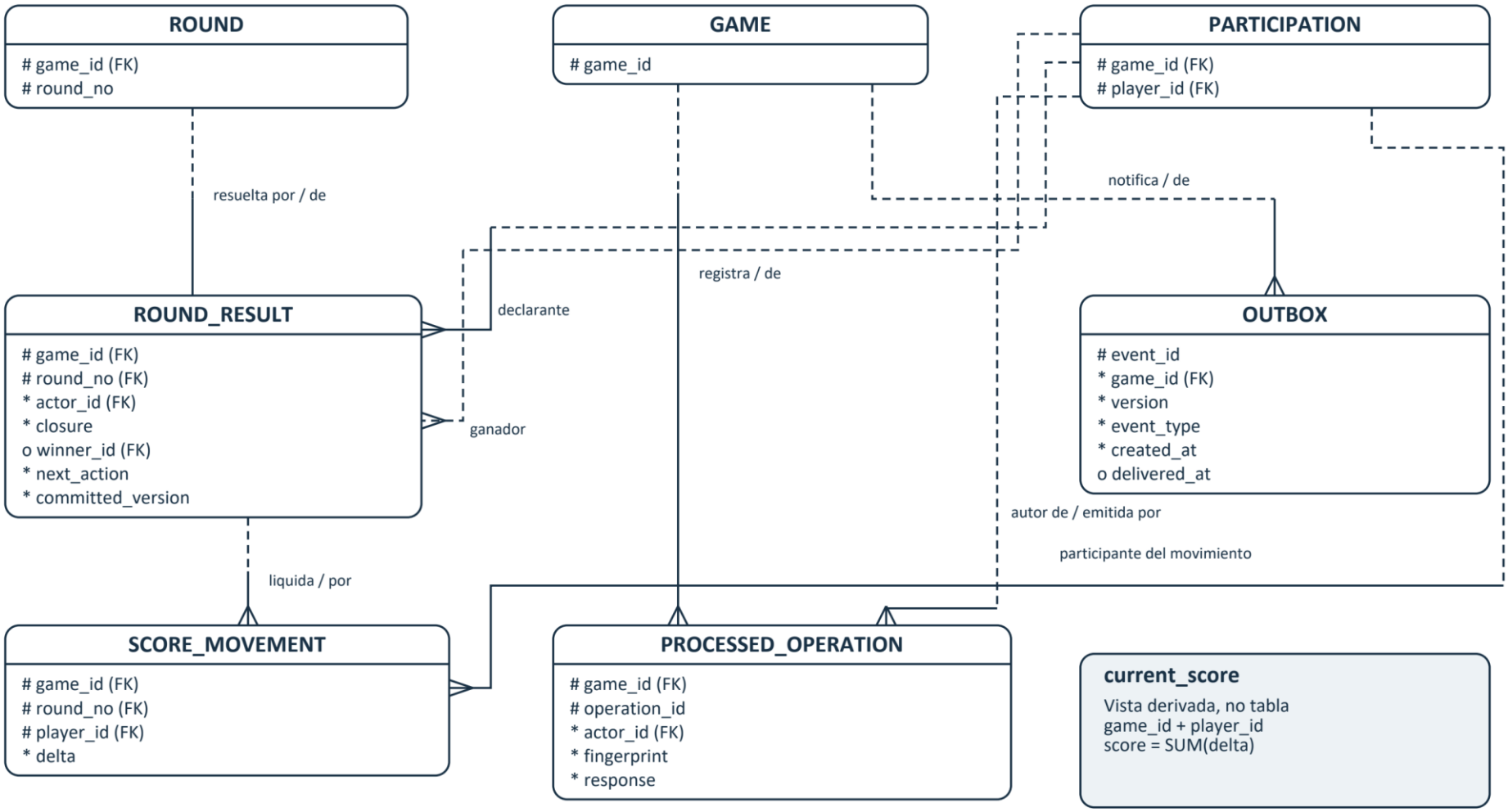
processed_operation.response contiene únicamente la respuesta serializada para repetir una operación. Es una proyección de transporte subordinada al modelo relacional; no se propone una segunda base documental ni una alternativa NoSQL paralela.

2.2 Modelo E-R Barker — mesa y rondas



Barker: # identificador; * obligatorio; o opcional. Mitad discontinua: participación opcional; pata de cuervo: muchos.

2.2 Modelo E-R Barker — liquidación e historial



Las entidades superiores muestran solo su identificador; sus atributos completos están en la vista de mesa.

2.2 Normalización y restricciones del esquema

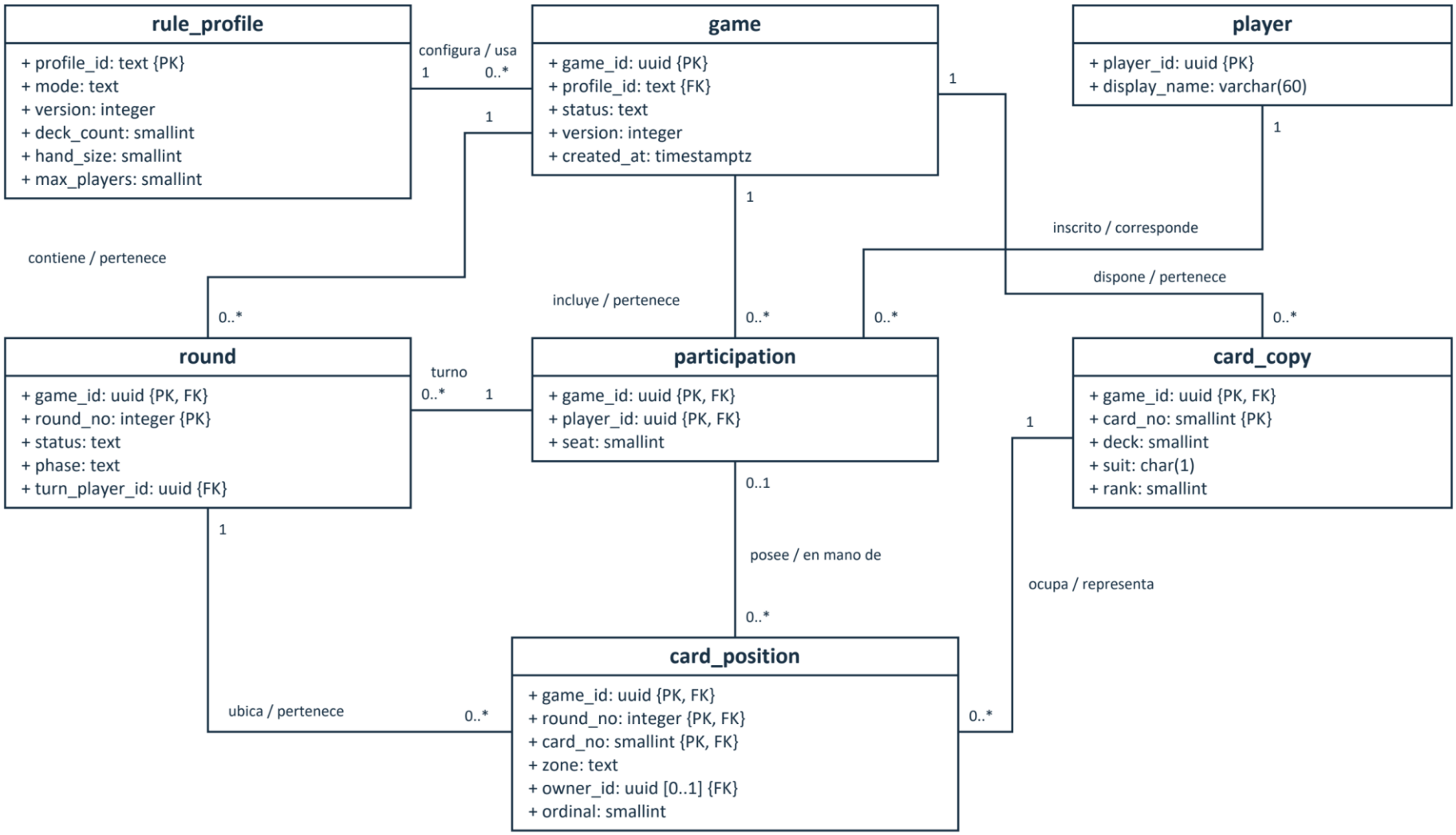
Relación	Clave y dependencia funcional principal
rule_profile	profile_id → modo, versión y parámetros. (mode, version) es clave candidata.
player / game	player_id → nombre. game_id → perfil, estado, versión y creación.
participation	(game_id, player_id) → seat. (game_id, seat) también identifica una participación.
round	(game_id, round_no) → estado, fase y jugador del turno.
card_copy	(game_id, card_no) → mazo, palo y rango. Segunda clave: partida + mazo + palo + rango.
card_position	(game_id, round_no, card_no) → zona, dueño y orden. La posición dentro de una zona también es única.
round_result	(game_id, round_no) → declarante, cierre, ganador, siguiente acción y versión confirmada.
score_movement	(game_id, round_no, player_id) → delta. No se almacena nombre ni total acumulado.
processed_operation	(game_id, operation_id) → actor, fingerprint y respuesta.
outbox	event_id → partida, versión, tipo y fechas. (game_id, version, event_type) es clave candidata.

Primera forma normal: cada columna representa un valor del contrato; los movimientos y las ubicaciones son filas, no listas incrustadas. Segunda: los atributos no clave dependen de toda la clave compuesta. Tercera: nombres, perfiles y cartas no se duplican en los movimientos; no hay dependencias transitivas de atributos no clave en las tablas autoritativas.

current_score calcula SUM(delta) por participación, con cero inicial. Los valores antes y después de una liquidación se calculan al construir su resultado. Así no se mantienen tres copias independientes del mismo acumulado. La respuesta JSON conserva la representación emitida para idempotencia, no se consulta como fuente del marcador.

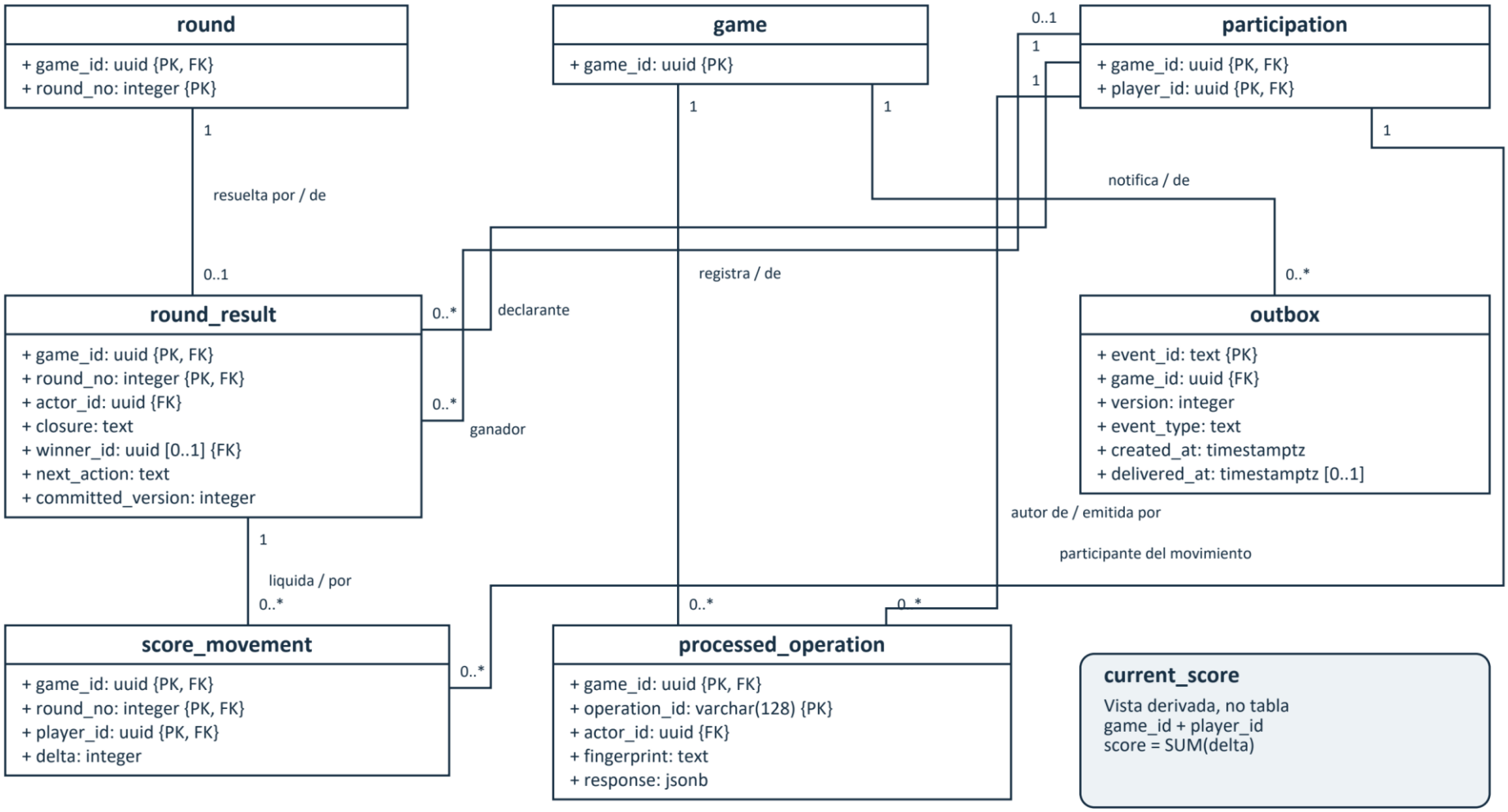
El SQL exige una sola ronda abierta por partida, un acta por ronda, un movimiento por jugador y ronda, un asiento por participante y una ubicación por copia. Perfil y asociación de perfil a partida son inmutables. La creación de 104 copias, el mínimo de dos jugadores y la preparación de la ronda se validan en la operación de inicio; las claves por sí solas no garantizan esos mínimos.

2.3 Diagrama UML de Clases de Entidad y Mapeo — mesa



Cada clase corresponde a una tabla homónima. Tipos SQL explícitos; asociaciones por PK/FK compuestas.

2.3 Diagrama UML de Clases de Entidad y Mapeo — liquidación



Las entidades superiores muestran solo su identificador; sus atributos completos están en la vista de mesa.

2.4 Estrategia de Transaccionalidad y Consistencia

CloseRound.execute recibe partida, ronda, identidad autenticada y comando. El adaptador abre una transacción READ COMMITTED y toma SELECT FOR UPDATE sobre la fila game. Todos los futuros comandos que modifiquen esa partida deben seguir la misma disciplina de bloqueo.

Paso	Operación e invariante
1. Bloquear y leer	Bloquear la partida, cargar la ronda y obtener manos y acumulados consistentes. Bloqueos siempre en el mismo orden.
2. Autorizar	Comprobar membresía y turno. Consultar operationId antes de rechazar la versión de un reintento ya confirmado.
3. Deduplicar	Si existe, comparar actor y fingerprint; devolver la respuesta guardada. Reutilización con otra acción: conflicto.
4. Validar	Exigir ronda abierta, fase de decisión y versión esperada. Evaluar la estrategia fijada en el perfil.
5. Liquidar	Conservar el descuento que permitió aceptar el toque. Calcular eliminaciones, ganador y siguiente acción.
6. Persistir	Insertar acta y movimientos; cerrar ronda; actualizar versión y estado; guardar operación y evento pendiente.
7. Confirmar y notificar	COMMIT antes de enviar. El despachador procesa la outbox y marca entrega después de publicar.

Si falla cualquier escritura, ROLLBACK conserva el estado previo. Si falla la red después del commit, la outbox permite reintentar. La entrega puede repetirse: eventId sirve para deduplicar y la versión impide retroceder el estado del cliente. No se promete entrega exactamente una vez.

Ante dos cierres, el segundo espera el bloqueo y observa la nueva versión. Un reintento de la misma operación obtiene la misma acta; una operación distinta encuentra la ronda cerrada. Las PK y restricciones únicas actúan como una segunda barrera.

La prueba PostgreSQL induce una falla al insertar la outbox, después de escribir acta y puntos, y comprueba la reversión. También dispara dos operaciones concurrentes mediante conexiones distintas. Las pruebas de memoria sirven para el dominio; la prueba de PostgreSQL verifica el mecanismo real de bloqueo.

3. Registro Formal de Decisiones Arquitectónicas (ADRs)

ADR-03 Estilo arquitectónico y extensibilidad de reglas

Estado: adoptado para el diseño del Entregable 2. Implementación parcial comprobable en design/src. Responsable: Juan José Alzate Rojas. Fecha de decisión: 27 de octubre de 2026.

Contexto

La plataforma ofrece Clásica de una ronda y Apuntado de rondas acumulativas. Configurability, Testability y Responsiveness son el Top 3 del Entregable 1. Las reglas cambian con más frecuencia que la coordinación de turno y cierre. Se requiere añadir variantes sin modificar ese núcleo y sostener la entrega con un solo desarrollador.

Decisión

Organizar el backend como monolito modular por capacidades; el frontend tiene un despliegue independiente. CloseRound coordina la transacción a través de RoundStore e invoca Variant mediante VariantCatalog. Las implementaciones classic y apuntado dependen únicamente de contratos y utilidades puras de dominio; no se importan entre sí. bootstrap/catalog.ts es el punto de composición. Añadir una variante requiere implementar Variant, registrarla, publicar un perfil versionado y desplegar el conjunto. No se modifica CloseRound ni se ofrecen plug-ins descargables en ejecución.

Alternativas y fundamento

Microkernel permite un registro y ciclo de vida de extensiones independientes; implica compatibilidad binaria o de contratos, carga, aislamiento y gestión de fallas de plug-ins. No hay terceros que distribuyan reglas ni necesidad de reemplazarlas en caliente. El monolito modular proporciona el aislamiento necesario mediante interfaces y verificaciones de imports, conservando llamadas locales y una transacción SQL. Un monolito con condicionales dispersos reduce el código inicial, pero dificulta mantener dos modalidades y sus pruebas.

Consecuencias

Positivas: evaluadores puros, una frontera transaccional, selección por versión y extensión por contrato. Negativas: despliegue conjunto, posible consumo compartido de CPU y necesidad de vigilar los límites. Se admite reiniciar para publicar variantes; las partidas ya iniciadas conservan su perfil. El beneficio de respuesta es una expectativa arquitectónica, no un resultado de una prueba multijugador.

Compliance

Ejecutar npm run test:architecture: cero dependencias cruzadas entre variantes, cero ciclos y matriz de imports permitidos. FF-05 impide infraestructura en dominio, reglas y aplicación. Ejecutar tests/domain.test.ts: una estrategia sintética se incorpora por inyección sin modificar CloseRound; versión desconocida y registro duplicado se rechazan. Revisar el diff del núcleo al incorporar una nueva variante. Evidencia en docs/evidence/verification.json.

Notas y revisión

Amplía ADR-01 y ADR-02 del Entregable 1; no los sustituye. Revisar si aparece una necesidad demostrada de instalar reglas por terceros o desplegarlas con independencia. Las configuraciones publicadas son inmutables y requieren otra versión para cambiar.

3. ADR-04

ADR-04 Persistencia relacional centralizada en PostgreSQL

Estado: adoptado para el diseño del Entregable 2. Responsable: Juan José Alzate Rojas. Fecha: 27 de octubre de 2026.

Contexto

Una partida acumula rondas, participantes y movimientos. Un cierre aceptado debe registrar juntos acta, penalizaciones, versión, respuesta idempotente y evento pendiente. Dos copias de una carta pueden tener el mismo palo y rango si pertenecen a mazos distintos; una misma copia solo ocupa una ubicación por ronda.

Decisión

Una instancia PostgreSQL 18 centralizada, accesible desde el adaptador de persistencia del servidor. Modelo relacional en 3FN para datos autoritativos; PK/FK compuestas mantienen pertenencia a partida. El puntaje se deriva de score_movement mediante current_score. processed_operation.response guarda una respuesta JSON de transporte para repetirla, no un documento autoritativo del juego. outbox contiene metadatos de eventos, sin manos privadas.

Alternativas

SQLite facilita ejecutar un solo proceso, pero limita la concurrencia de escritores y exige revisar el despliegue cuando varios procesos comparten estado. Una base documental permite incrustar una partida y escribirla de forma atómica; el historial creciente, las referencias a jugadores y la duplicación de información exigen reglas adicionales. PostgreSQL coincide con la tecnología del prototipo y ofrece restricciones explícitas para el historial y la liquidación.

Consecuencias

Positivas: transacciones locales, consultas de historial, claves de pertenencia y acumulados reproducibles. Negativas: servicio de base de datos, migraciones, respaldos y contención por partida. Los bloqueos serializan comandos dentro de una partida; distintas partidas se procesan independientemente. La centralización no ofrece alta disponibilidad por sí sola.

Compliance

Ejecutar database/schema.sql en una base vacía. tests/schema.test.ts verifica DDL, PK, FK, dos copias de carta, ronda abierta única, inmutabilidad del perfil y suma de movimientos. tests/postgres.test.ts, contra una base efímera, verifica cierres concurrentes con dos conexiones y rollback si falla la outbox. La prueba PGlite local no sustituye la verificación de bloqueos entre conexiones. Revisar migraciones y la correspondencia entre UML, Barker y SQL.

Notas y revisión

La prueba PostgreSQL requiere ARCH_TEST_DATABASE_URL apuntando a una base exclusiva y vacía; CI proporciona ese servicio. La identidad de sesión permanece fuera del esquema académico del juego y será integrada con autenticación. Revisar topología ante contención sostenida o requisitos de disponibilidad nuevos.

3. ADR-05

ADR-05 Llamadas locales síncronas y notificación posterior al commit

Estado: adoptado para el diseño del Entregable 2. Responsable: Juan José Alzate Rojas. Fecha: 27 de octubre de 2026.

Contexto

Validar un cierre, impedir la autoeliminación por toque y liquidar puntos requieren un resultado coherente inmediato. La comunicación con los jugadores puede fallar después del commit. WebSocket es el canal de sincronización requerido; no proporciona por sí solo autorización, orden de negocio o entrega durante desconexiones.

Decisión

Usar llamadas locales mediante interfaces en el camino crítico: coordinador, registro, estrategia y unidad de trabajo. Guardar estado, resultado, movimientos, respuesta de operación y evento de outbox en una misma transacción. Un despachador desacoplado procesa la outbox después del commit. La implementación del publicador WebSocket enviará vistas autorizadas, autenticará suscripciones y repondrá una instantánea al reconectar. No se utilizará polling ni long polling de estado en los clientes.

Alternativas y fundamento

Eventos locales volátiles para toda la liquidación desacoplan llamadas, pero complican orden, errores y recuperación; una caída puede perder el aviso. Eventos durables de cada subpaso añaden coordinación innecesaria para una transacción local. Llamadas síncronas para enviar por red dentro de la transacción prolongan el bloqueo y mezclan disponibilidad del cliente con consistencia del marcador. La solución mixta concentra invariantes y desacopla solo la notificación.

Consecuencias

Entrega al menos una vez: un fallo después del envío y antes de marcar entrega puede producir duplicados. El cliente deduplica por eventId y aplica versiones de forma monótona; un salto pide una instantánea por WebSocket. operationId identifica reintentos y un fingerprint rechaza su reutilización con otro contenido. La outbox tiene costo de almacenamiento y requiere reintentos con límite de frecuencia, observabilidad y limpieza. Un worker puede consultar la outbox en el servidor; eso no constituye polling del juego desde el cliente.

Compliance

Las pruebas de aplicación comprueban un solo resultado ante reintento, rechazo de versión antigua y de cambio de payload, no revelación ante toque rechazado, rollback y retención del evento si el publicador falla. La prueba PostgreSQL verifica la atomicidad real. El adaptador WebSocket y los ensayos con seis dispositivos son trabajo de integración del MVP: se verificará privacidad, reconexión y las metas p95 del Entregable 1. No se declaran como mediciones realizadas por esta base de código.

Notas e historial

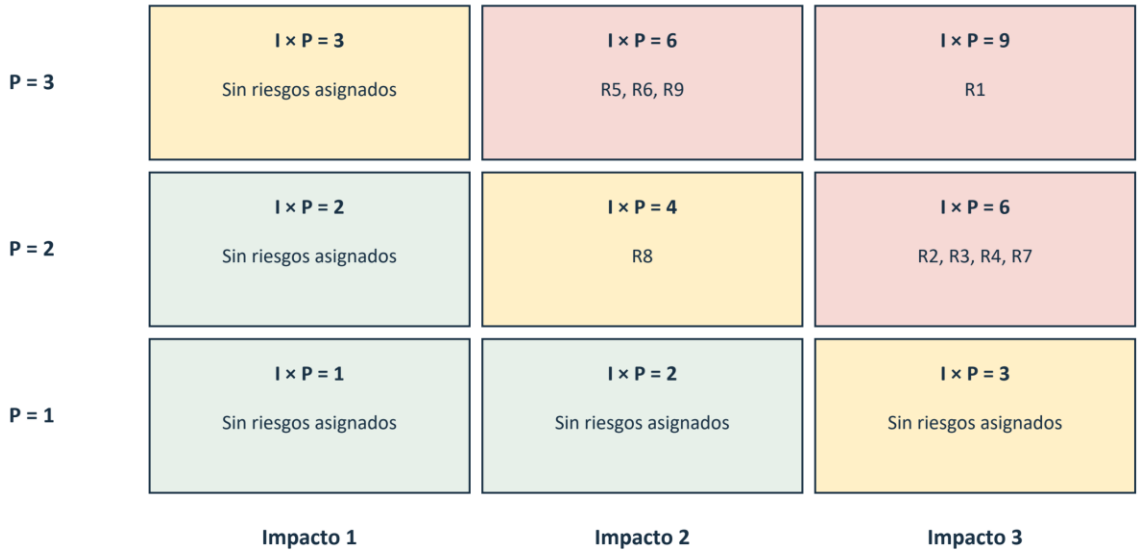
Integra y amplía el antecedente E1-ADR-03 sobre WebSockets, conservado en docs/history/E1-ADR-03-websockets.md. En el Entregable 2, ADR-03 se reserva para estilo y extensibilidad por exigencia de la guía. La comunicación interna y el transporte hacia clientes son decisiones relacionadas, pero distintas.

4. Análisis de Riesgo Arquitectónico y Risk Storming

4.1 Matriz de Riesgo 3x3

La prioridad se calcula como impacto × probabilidad. Son valoraciones cualitativas del escenario de diseño; no son frecuencias observadas ni porcentajes estadísticos. Una misma escala se aplica antes y después de proponer mitigaciones.

Valor	Probabilidad	Impacto
1	Poco probable; exige una condición excepcional o varias fallas simultáneas.	Efecto local y recuperable, sin cambiar puntuación ni exponer información.
2	Plausible durante una sesión o un cambio de código.	Interrumpe una ronda o dificulta seriamente mantenimiento y respuesta.
3	Esperable por reintentos, concurrencia o cambios frecuentes sin controles.	Altera ganador o marcador, expone manos o compromete la integridad de la partida.



Bajo 1–2 | Medio 3–4 | Alto 6–9. Valoración cualitativa individual.

Los identificadores R1–R9 se mantienen en el registro, las mitigaciones y sus pruebas. Las zonas rojas requieren un cambio arquitectónico o una restricción verificable antes de liberar la funcionalidad afectada.

4.2 Reporte de Risk Storming

Este entregable se desarrolla individualmente debido a mi condición de estudiante de prácticas y a las limitaciones para asistir a la mayoría de las clases, situación que comuniqué previamente al profesor Fernan por correo. Por ello, el ejercicio de Risk Storming recoge únicamente mi perspectiva.

Fase 1 Identificación individual

El análisis recorre las fronteras cliente-servidor, coordinador-reglas, aplicación-persistencia y commit-notificación. Para cada frontera se consideran entradas inválidas, repetición, concurrencia, falla parcial y exposición de información. Se revisan también las diferencias entre modalidades y el mantenimiento de los validadores.

Grupo de escenarios	Riesgos identificados
Integridad y concurrencia	R1 doble liquidación; R4 mezcla de reglas; R7 comandos no autorizados u obsoletos.
Privacidad y comunicación	R2 exposición de manos; R3 estado confirmado sin notificación recuperable.
Evolución y capacidad	R5 ciclos; R6 complejidad; R8 evaluación lenta; R9 puntos ciegos de revisión individual.

Fase 2 Consenso — adaptación individual

No hubo consenso grupal ni votación de otros integrantes. Esta fase se adapta a una segunda revisión individual: se agrupan duplicados, se aplica la misma escala y se contrasta cada valoración con su consecuencia concreta y los controles existentes. La justificación queda registrada por riesgo para permitir su revisión posterior.

Se priorizan los escenarios que afectan puntuación, privacidad o autoridad del servidor. R8 permanece medio porque la carga real aún no está medida. La ausencia de revisión por pares se registra como R9; una comprobación automática no reemplaza esa perspectiva adicional.

Fase 3 Mitigación

Las respuestas se incorporan al diseño: bloqueo e idempotencia, perfiles inmutables, proyección privada, outbox y restricciones de dependencias. Cada mecanismo se vincula con una prueba o con una verificación de integración pendiente. El riesgo residual es una estimación condicionada a mantener esos controles, no una garantía de eliminación del riesgo.

Responsable de la valoración, del registro y del seguimiento: Juan José Alzate Rojas. La base de riesgos editable se conserva en docs/risk-storming/risks.json.

4.3 Plan de Mitigación de Riesgos Críticos

R1 Doble liquidación por reintento o cierres simultáneos

Inicial: P=3, I=3, nivel=9. Hay varios clientes y los reintentos son esperables; repetir un descuento altera el ganador.

Mitigación: Bloqueo por partida, versión esperada, PK del acta y clave de operación con fingerprint.

Verificación: Pruebas de aplicación y PostgreSQL entre conexiones. Residual estimado: P=1, I=3, nivel=3.

R2 Exposición de manos privadas

Inicial: P=2, I=3, nivel=6. Una difusión común puede incluir datos de otro jugador; compromete la equidad de toda la ronda.

Mitigación: Proyección por destinatario, autorización de sala y outbox sin cartas.

Verificación: Prueba de vista privada; integrar prueba de dos sesiones WebSocket. Residual estimado: P=1, I=3, nivel=3.

R3 Estado confirmado sin notificación recuperable

Inicial: P=2, I=3, nivel=6. El commit y el envío por red son acciones distintas; una caída intermedia desincroniza clientes.

Mitigación: Outbox transaccional, eventId estable, reintento y snapshot versionado al reconectar.

Verificación: Fallo de outbox revierte; fallo de publicador conserva evento. Residual estimado: P=1, I=3, nivel=3.

R4 Reglas de una modalidad aplicadas a la otra

Inicial: P=2, I=3, nivel=6. Se comparten cartas y turno, pero el fin de partida y los puntos son distintos.

Mitigación: Variant inyectada por clave y versión; perfiles inmutables; pruebas por modalidad.

Verificación: Clásica vuelve a inicio; Apuntado liquida y continúa. Residual estimado: P=1, I=3, nivel=3.

4.3 Mitigación estructural y seguimiento

R5 Acoplamiento y ciclos entre módulos

Inicial: P=3, I=2, nivel=6. Un único proceso permite imports fáciles; la duplicación del motor encarece los cambios.

Mitigación: Contratos comunes, matriz de imports, raíz de composición y TSArch en cada cambio.

Verificación: FF-01 a FF-05 y controles negativos. Residual estimado: P=1, I=2, nivel=2.

R6 Validadores demasiado complejos

Inicial: P=3, I=2, nivel=6. As, duplicados, patrones y toque introducen muchas ramas y casos límite.

Mitigación: Funciones puras pequeñas, particionador compartido y puerta CC máxima 10.

Verificación: FF-06, FF-08 y reporte real de complejidad. Residual estimado: P=1, I=2, nivel=2.

R7 Comando de otro jugador o de versión obsoleta

Inicial: P=2, I=3, nivel=6. El cliente no es confiable y puede reenviar una acción después de perder su turno.

Mitigación: Identidad desde sesión; membresía, turno, fase y versión dentro de la unidad de trabajo.

Verificación: Rechazo de intruso, turno incorrecto y versión antigua. Residual estimado: P=1, I=3, nivel=3.

R8 y R9 Riesgos que requieren seguimiento

R8: P=2, I=2, nivel 4. Se mantiene residual 4 hasta medir CPU y latencia bajo carga. R9: P=3, I=2, nivel 6. El recorrido sistemático de escenarios y la revisión de resultados reducen la estimación a P=2, I=2, nivel 4; permanece la limitación de una sola perspectiva.

Se reabre el análisis cuando cambia una variante, una transacción, un contrato o la topología. Un resultado rojo sin mitigación bloquea la liberación de la funcionalidad afectada. La aceptación de un riesgo residual debe registrar motivo, responsable y condición de revisión.

5. Gobernanza y Funciones de Idoneidad (Fitness Functions)

5.1 Reglas de Gobernanza

Regla	Comprobación y criterio	Respuesta al incumplimiento
G1 Aislamiento	UI solo usa contratos; no importa persistencia ni drivers SQL. FF-01, FF-04 y FF-05.	Reubicar acceso detrás de un puerto. La verificación falla.
G2 Independencia	Cero ciclos de archivos y cero imports entre classic y apuntado. FF-02 y FF-03.	Extraer contratos o lógica compartida al dominio puro.
G3 Complejidad	CC clásica de McCabe ≤10 por función en domain, variants y application. FF-06.	Descomponer responsabilidades conservando pruebas de comportamiento.
G4 Núcleo estable	Aplicación no importa variantes concretas ni infraestructura. FF-04.	Registrar implementaciones únicamente en bootstrap.
G5 Perfil inmutable	Versión fijada en partida; rechazo de modificación de perfil publicado o de su asociación.	Crear una versión nueva para partidas futuras.
G6 Atomicidad	Acta, movimientos, operación y outbox se confirman o revierten juntos.	Bloquear integración si falla la prueba transaccional.

Juan José Alzate Rojas mantiene decisiones y excepciones. Cada cambio que atraviese un límite requiere actualizar el ADR pertinente y ejecutar el pipeline. Las excepciones deben identificar alcance, razón, responsable y revisión; no se aceptan desactivaciones silenciosas de pruebas o del límite de complejidad.

Estas funciones son automáticas y se ejecutan por cambio en CI. Comprueban propiedades estructurales y algunas invariantes del cierre. La latencia multijugador y la autorización del transporte requieren ensayos adicionales cuando el adaptador WebSocket esté integrado.

5.2 Código Fuente de las Fitness Functions

Se utiliza un solo stack: TypeScript con el ejecutor de pruebas de Node, TSArch y ESLint. El código completo está en tests/architecture/fitness.test.ts. Se usan patrones anclados porque los paths del grafo son relativos al tsconfig; esto evita selectores que no encuentren archivos y aprueben sin comprobar nada.

```
const files = () => filesOfProject(CONFIG);

test('UI sin acceso a persistencia', async () => {
  const result = await files()
    .matchingPattern('^design/src/ui/.*')
    .shouldNot().dependOnFiles()
    .matchingPattern('^design/src/persistence/.*').check();
  assert.deepEqual(result, []);
});

test('Cero ciclos', async () => {
  const result = await files()
    .matchingPattern('^design/src/.*')
    .should().beFreeOfCycles().check();
  assert.deepEqual(result, []);
});

test('Variantes independientes', async () => {
  for (const [from, to] of [
    ['classic', 'apuntado'], ['apuntado', 'classic']
  ]) {
    const result = await files()
      .matchingPattern(`^design/src/variants/${from}/.*`)
      .shouldNot().dependOnFiles()
      .matchingPattern(`^design/src/variants/${to}/.*`)
      .check();
    assert.deepEqual(result, []);
  }
});
```

La matriz completa permite contracts como base común; domain solo depende de contracts; application no depende de variantes concretas; variants depende de domain y contracts. persistence y synchronization implementan puertos. bootstrap compone las implementaciones.

Además se examinan las dependencias externas prohibidas y se exige que existan los módulos esperados. Las pruebas se aplican a design/src. El repositorio académico contiene esta base de diseño; el backend anterior se mantiene en un repositorio independiente.

5.2 Evidencia ejecutada y controles negativos

Verificación	Resultado y alcance
Compilación TypeScript	Sin errores en la base de diseño y las pruebas.
Reglas y aplicación	16 pruebas aprobadas: as, duplicados, cierres, toque, umbrales, permisos, reintentos, rollback y extensión.
Arquitectura	9 pruebas aprobadas, incluidos controles de alcance y casos negativos.
SQL local	1 prueba integral aprobada con PGlite: DDL, claves, restricciones, acumulados y rollback.
PostgreSQL en CI	Prueba de cierre con dos conexiones y falla inducida de outbox aprobada en PostgreSQL 18.
Complejidad	55 funciones medidas; máximo 8; umbral 10.
Reproducción	Versiones fijadas en package-lock.json; Node 24; configuración y comandos incluidos.

Demostración de sensibilidad de las pruebas

Las muestras deliberadamente inválidas se alojan en tests/fixtures/invalid y no forman parte del código de producción. Una UI importa persistencia, dos archivos forman un ciclo y una variante importa la otra. La prueba exige encontrar infracciones en cada muestra.

```
const invalid = () => filesOfProject(FIXTURE_CONFIG);
const violations = await invalid()
  .matchingPattern('^ui/.*').shouldNot().dependOnFiles()
  .matchingPattern('^persistence/.*').check();
assert.ok(violations.length > 0);

// Otro control confirma que un import real UI -> contracts
// sí es encontrado por el selector utilizado en la prueba.
```

El control de complejidad genera una función con CC 11 y exige un error; otra con CC 10 debe aprobar. Así se verifica el límite inclusivo. La idempotencia también se comprueba después de serializar el resultado como JSON, tal como ocurre en persistencia.

Los archivos verification.json y complexity.json contienen resultados locales y medidas. La ejecución remota se conserva en GitHub Actions. Estas evidencias no equivalen a una prueba de carga con seis navegadores ni a la validación final de la experiencia de usuario.

5.3 Verificación de Complejidad Ciclomática

ESLint analiza la complejidad clásica de McCabe por función. Se aplica a domain, variants y application, incluyendo los validadores y la coordinación del cierre. No se utiliza un promedio por archivo: una sola función que supere 10 hace fallar la comprobación.

```
import tseslint from 'typescript-eslint';
export default [{
  files: [
    'design/src/domain/**/*.ts',
    'design/src/variants/**/*.ts',
    'design/src/application/**/*.ts'
  ],
  languageOptions: { parser: tseslint.parser },
  rules: {
    complexity: ['error', { max: 10, variant: 'classic' }]
  }
}];
```

Función o ubicación	CC medida
design/src/variants/apuntado/rules.ts:4	8
design/src/variants/apuntado/rules.ts:13	7
design/src/domain/cards.ts:4	6
design/src/domain/cards.ts:8	5
design/src/domain/cards.ts:18	5

scripts/complexity-report.ts obtiene los valores desde la misma regla de ESLint y conserva archivo y línea. La ejecución de aceptación usa max=10; la extracción de métricas usa max=0 para obtener el valor de todas las funciones y luego comprueba el umbral. No se emplean constantes que simulen una medición.

Integración en el pipeline

GitHub Actions instala desde el lockfile, comprueba tipos, ejecuta dominio, arquitectura y esquema, verifica CC y genera métricas. Después ejecuta la prueba contra PostgreSQL 18 en un servicio efímero. Cualquier proceso con código distinto de cero hace fallar el job; las evidencias se publican incluso si alguna etapa falla.

5.3 Reproducción y alcance de la entrega

```
git clone \
  https://github.com/AlzateJJ/apuntado-arquitectura.git
cd apuntado-arquitectura
# Usar Node.js 24
npm ci --ignore-scripts
npm run verify
```

La prueba de PostgreSQL utiliza ARCH_TEST_DATABASE_URL y una base exclusiva y vacía. En CI el servicio se crea y elimina para cada ejecución. Los comandos locales de verificación no usan la base ni el archivo .env del prototipo.

Artefacto	Ubicación en el repositorio
Modelos UML y Barker	docs/models/: drawio, SVG, PDF vectorial, PNG y entities.json.
Decisiones	docs/adrs/ADR-01…ADR-05; antecedente de WebSockets en docs/history/.
Información	database/schema.sql; tipos y relaciones en los modelos de entidades.
Riesgos	docs/risk-storming/risks.json y reporte individual de este documento.
Fitness functions	tests/architecture/fitness.test.ts y eslint.architecture.config.mjs.
Código verificado	design/src/; pruebas de reglas, aplicación, SQL y PostgreSQL en tests/.
Integración continua	Workflow architecture.yml; resultados en docs/evidence/ y GitHub Actions.

La entrega comprueba la estructura modular y el camino de cierre de ronda. Quedan para integrar el MVP: sala y reparto completos, robo y descarte, autenticación del transporte, servidor WebSocket, reconexión entre dispositivos y pantallas de juego. La navegación de partida única ya está expresada y probada mediante el resultado home, cuyo destino es la vista principal.

Se conservan como metas del primer entregable evaluación p95 ≤50 ms y confirmación y difusión p95 ≤500 ms en el escenario de piloto. Las pruebas actuales no constituyen la medición de esas metas. Su futura ejecución debe registrar carga, red, muestra y percentiles por separado.

El informe consolida el diseño, los riesgos y los controles. Los archivos fuente permiten modificar modelos y ejecutar comprobaciones; cualquier cambio posterior debe mantener alineados código, esquema, diagramas y ADR.